

ESP32 Ecosystem Overview, Communication Protocols, and Power Sizing

Report Title	<i>ESP32 Ecosystem Overview, Communication Protocols, and Power Sizing</i>
Program	<i>IoT Training Program</i>
Date	<i>21/7/2026</i>
Prepared By	Malak Alsharqawi
Organization	<i>Cyber Robot</i>

Contents

1.1 Fundamental Embedded Concepts.....	4
1.1.0 What Is an Embedded System, and Why Does This Chip Matter?	4
1.1.1 High-Level Hardware Packaging Hierarchy (Top-Down Approach).....	4
1.1.2 Deep Dive: Layer 1 — The Silicon Chip (System-on-Chip / SoC).....	5
Core Classification: Microcontroller (MCU) vs. Microprocessor (MPU)	6
Internal Silicon Functional Blocks	6
1.1.3 Deep Dive: Layer 2 — The PCB Module.....	7
Why Modules Are Necessary	7
MCU Modules vs. MPU Modules (System-on-Module / SoM)	8
WROOM vs. WROVER Module Distinction	8
1.1.5 Processor Instruction Set Architectures (ISA)	10
Why the Semiconductor Industry is Shifting to RISC-V	11
1.1.6 Native USB Hardware Capabilities.....	11
1.1.7 Key Terms at a Glance	13
1.1.8 Section Recap.....	13
1.2 The ESP32 Ecosystem Overview	14
1.2.0 Introduction: What Is the ESP32 and Why Use It?	14
1.2.1 Board Survey & Core ESP32 Variants.....	15
1.2.2 Ecosystem Comparison Matrix	16
1.2.3 Pros & Cons: ESP32 vs. Traditional Microcontrollers.....	17
Section 2: Communication Protocols Deep-Dive.....	19
2.1 UART (Universal Asynchronous Receiver-Transmitter).....	19
2.2 I ² C (Inter-Integrated Circuit)	21
2.3 SPI (Serial Peripheral Interface)	23
2.4 RS-485 (Industrial Differential Serial Communication).....	25
2.5 Summary Comparison Table	27
Section 3: Power Electronics & Sizing Fundamentals	28
3.1 Converters vs. Inverters (Power Electronics Fundamentals)	28
3.1.1 AC/DC Rectifiers (AC-to-DC Conversion)	28
3.1.2 DC/DC Converters — Buck (Step-Down) and Boost (Step-Up)	29
3.1.3 DC/AC Inverters (DC-to-AC Conversion).....	30

3.1.4 Summary: The Three Converter Families	31
3.2 Power Supply Sizing Guide (Step-by-Step Mathematical Example)	31
3.2.1 Why Steady-State Current Is the Wrong Number to Design Around	31
3.2.2 Hardware Architecture: Isolating Logic from the Motor Power Loop	32
3.2.3 Project 2 Load Inventory: Simple Motor Protection System	32
3.2.4 Step 1 — Sum the Simultaneous Peak Loads	33
3.2.5 Step 2 — Apply an Industrial Safety Margin (Headroom)	33
3.2.6 Step 3 — Select the Final Commercial Power Supply	33
3.2.7 Section 3 Recap	34
References	35

1.1 Fundamental Embedded Concepts

This section builds the vocabulary and mental model needed for everything that follows in this report. It is written for a reader who may be new to embedded hardware, so every technical term is explained in plain language the first time it appears, and each idea is reinforced with an everyday analogy before the formal engineering definition is given. By the end of this section, you should be able to look at a real ESP32 board and correctly name and explain every major part of it.

1.1.0 What Is an Embedded System, and Why Does This Chip Matter?

A general-purpose computer — a laptop or a phone — is built to run many different programs chosen by its user: a browser, a game, a spreadsheet. An embedded system is the opposite philosophy: it is a small computer built into a larger product to perform one dedicated job, usually invisibly. The controller inside a washing machine, a car's airbag sensor, a smart thermostat, and a Wi-Fi light bulb are all embedded systems. The ESP32 is one of the most popular chips used to build these systems today, primarily because it combines a capable processor with built-in Wi-Fi and Bluetooth on a single, inexpensive piece of silicon.

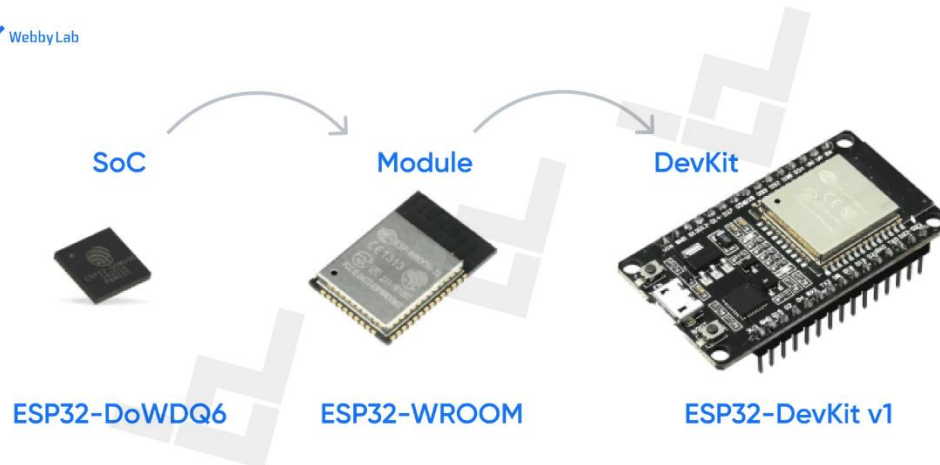
IN PLAIN TERMS

Think of a laptop as a Swiss Army knife — flexible, but bulky and expensive for a single job. An embedded chip like the ESP32 is more like a purpose-built tool: smaller, cheaper, and extremely good at its one task — such as reading a sensor and sending the result over Wi-Fi.

Understanding embedded hardware means understanding three things at once: the physical packaging you can see and touch, the silicon logic hidden inside that packaging, and the language (instruction set) that lets software talk to that silicon. The remainder of this section walks through all three, in that order.

1.1.1 High-Level Hardware Packaging Hierarchy (Top-Down Approach)

Every ESP32-based product a developer buys is really three products nested inside one another, like a set of Russian nesting dolls. Each outer layer exists purely to make the layer inside it more usable — smaller, more reliable, or easier to connect to.



- **Layer 1 - Silicon Chip (SoC):** the innermost doll. A tiny, fragile square of silicon a few millimetres wide, containing the actual processor. On its own it cannot be handled, soldered by hand, or plugged into anything.
- **Layer 2 - PCB Module:** the middle doll. The silicon chip is mounted on a small circuit board with just enough extra parts (memory, antenna, oscillator) to make it a complete, working radio-enabled computer — but still with no way to plug it into a USB port.
- **Layer 3 - Development Board (DevKit):** the outermost doll, and the one you actually hold in your hand. The module is mounted on a larger board with a USB port, power regulation, and pin headers, so it can be programmed from a laptop and plugged into a breadboard.

Reading the diagram from the inside out mirrors how the product is actually manufactured: the silicon die (soC) is manufactured first, then embedded inside a PCB module (Module), which is finally soldered onto a development board (DevKit). The sections below now examine each layer individually, working outward from the silicon, explaining exactly what is added - and why - at every step.

1.1.2 Deep Dive: Layer 1 — The Silicon Chip (System-on-Chip / SoC)

A **System-on-Chip (SoC)** is a single piece of silicon that packs a whole computer's worth of function — a processor plus its supporting circuitry - onto one chip, rather than spreading those parts across several separate chips on a board. The ESP32-D0WDQ6 (used in the classic ESP32) and the newer ESP32-S3 are both examples of an SoC die.



IN PLAIN TERMS

A generation ago, a computer's brain (CPU), memory, and I/O controllers were separate chips wired together on a board — like separate kitchen appliances on a counter. An SoC squeezes all of

those appliances into a single compact unit, the way a modern combi-oven bundles a microwave, convection oven, and grill into one box.

Everything a chip is capable of doing is permanently fixed the moment it is manufactured — nothing can be physically added to a die afterward. This is why the very first and most important question to ask about any chip is what was actually built into it. That question leads directly to the MCU vs. MPU classification below.

Core Classification: Microcontroller (MCU) vs. Microprocessor (MPU)

This is one of the most important distinctions in embedded electronics, and it is decided entirely at the silicon level by what is physically integrated onto the die, not by anything added later on a board.

ANALOGY

An MCU is like a fully furnished studio apartment: it comes with a bed, a kitchen, and storage already built in, so you can move in and live immediately. An MPU is like an empty, unfurnished apartment with great architecture but no furniture — livable only after you bring in a bed (RAM), a pantry (storage), and a power system (PMIC) yourself.

Aspect	Microcontroller (MCU)	Microprocessor (MPU)
What's on the die	CPU core + SRAM + Flash/peripheral controllers, all together	CPU core only (plus small cache)
External parts needed	Very few - often none beyond power	External RAM, external storage, and a PMIC are mandatory
Typical use case	Dedicated, real-time control tasks	General-purpose computing, running a full OS
ESP32 fits here?	Yes - the ESP32 family is an MCU	No
Example	ESP32, ESP32-S3, ESP32-C3	Raspberry Pi application processor

Because everything the ESP32 needs to run a program is already on the die, it can boot and execute code with almost no supporting circuitry, a defining trait of dedicated, real-time embedded control, and the reason MCUs like the ESP32 are the default choice for sensor nodes, actuators, and other single-purpose IoT devices.

Internal Silicon Functional Blocks

Opening up the die itself — figuratively — reveals several functional blocks that constantly cooperate to fetch, decode, and execute program instructions, millions or billions of times per second.

- **CPU Core:** the brain of the chip, made of four cooperating parts — the Arithmetic Logic Unit (ALU), which performs math and logic operations; general-purpose registers, which act as the CPU's tiny, instant-access scratchpad for values it is actively working with; the Program Counter (PC), which always points to the next instruction to run; and the Control Unit, which directs the whole fetch-decode-execute cycle, step by step, on every clock tick.
- **Internal SRAM:** SRAM (Static RAM) is fast, volatile memory — volatile meaning its contents vanish the instant power is removed. The classic ESP32 has approximately 520 KB of internal SRAM, used as the working scratchpad for the currently running program: its call stack (tracking which function called which) and its dynamically allocated variables (the heap).
- **Clock Oscillator & System Buses:** the clock oscillator is a tiny quartz-driven metronome that ticks at a fixed rate, pacing every synchronous operation on the die — nothing happens out of step with this beat. The Address, Data, and Control buses are the internal 'roads' that carry binary information between the CPU core, memory, and peripherals: the Address bus says where, the Data bus carries what, and the Control bus says what kind of operation (read or write) is happening.
- **Silicon Peripherals:** specialised hardware blocks that let firmware interact with the outside world without the CPU doing all the work manually. These include GPIO (General-Purpose Input/Output) pins for reading switches or driving LEDs, ADC/DAC converters for translating between analog voltages and digital numbers, timers for precise time measurement, PWM (Pulse-Width Modulation) generators for dimming LEDs or driving motors, and hardware serial controllers — UART, SPI, and I2C — that handle the strict timing of serial communication so the CPU doesn't have to.

WHY THIS MATTERS

Every peripheral listed above exists so the CPU core can delegate repetitive, timing-sensitive work to dedicated hardware. Without a hardware UART controller, for example, the CPU would have to toggle a pin at exactly the right microsecond intervals itself — wasting nearly all of its processing power just to send text over a serial cable.

1.1.3 Deep Dive: Layer 2 — The PCB Module

A **PCB module** — for example the ESP32-WROOM-32 or ESP32-WROVER — takes the bare SoC die from Layer 1 and mounts it on its own small printed circuit board, adding the minimum extra parts required to turn a fragile silicon chip into a complete, reliable, radio-capable component that a hardware designer can actually solder onto a larger product.

Why Modules Are Necessary

It might seem simpler to solder the bare SoC die directly onto a product's main board, skipping the module step entirely. In practice this is avoided, for three concrete engineering reasons:

-
- **RF Compliance:** any product that transmits radio signals must be tested and certified by regulators — FCC in the United States, CE in Europe — before it can legally be sold. A module is pre-certified once by its manufacturer, so any product built around it inherits that certification instead of repeating expensive lab testing from scratch.
 - **Impedance Matching:** for a radio signal to leave the antenna efficiently rather than reflecting back and wasting power, every trace along its path must be tuned — impedance-matched — with sub-millimetre precision for the 2.4 GHz Wi-Fi/Bluetooth band. Module manufacturers with specialised RF engineering expertise solve this once, correctly, so individual product designers do not have to repeat delicate RF layout work themselves.
 - **Shielding:** a grounded metal RF can — the small metal box visible on top of most ESP32 modules — physically covers the sensitive radio circuitry, blocking electromagnetic interference from leaking in or out and disrupting nearby components.

MCU Modules vs. MPU Modules (System-on-Module / SoM)

The amount of extra circuitry a module needs to add depends entirely on whether the silicon chip inside it is an MCU or an MPU, following directly from the distinction explained in Section 1.1.2.

- **MCU Module (e.g., ESP32-WROOM):** since the underlying MCU die already contains its own SRAM and peripheral logic, the module only needs to add SPI Flash memory (to store the actual program, since internal SRAM alone is volatile and too small), a crystal oscillator for precise timing, a handful of passive components (resistors and capacitors), and a small PCB antenna.
- **MPU Module / System-on-Module (e.g., Raspberry Pi Compute Module):** because the underlying MPU die has no internal RAM or storage at all, its module-level sub-PCB must integrate the MPU chip together with discrete DDR RAM chips, eMMC Flash storage, and a PMIC (Power Management IC) chip to regulate several different voltage rails. The result is a noticeably larger, more complex, and more expensive module than an MCU module.

WROOM vs. WROVER Module Distinction

While both **ESP32-WROOM** and **ESP32-WROVER** module families are powered by the same underlying ESP32 System-on-Chip (SoC) giving them identical dual-core processing capabilities and wireless functionality they differ fundamentally in memory capacity, physical dimensions, and pin multiplexing:

- **Integrated PSRAM Expansion:** The defining feature of the **WROVER** series is the integration of an additional **4 MB to 8 MB of external Pseudo-Static RAM (PSRAM / SPIRAM)** inside the module casing, connected via high-speed SPI/QSPI. The **WROOM** series lacks PSRAM and relies solely on the SoC's internal 520 KB SRAM.
- **GPIO Pin Availability:** On WROVER modules, **GPIO16 and GPIO17** are hardwired internally to manage the PSRAM chip. Consequently, these two pins cannot be used as general-

purpose I/O or secondary UART pins. On WROOM modules, GPIO16 and GPIO17 remain fully available.

- **Physical Footprint:** Due to the additional PSRAM die packaged inside the metal shield, WROVER modules are slightly longer (~31.4 mm vs. ~25.5 mm) than standard WROOM modules.

IN PLAIN TERMS

The internal 520 KB SRAM on the ESP32 chip is like a **small desk surface**—plenty of room for day-to-day paperwork.

The PSRAM on a **WROVER** module is like renting a **massive side table** placed right next to that desk. Accessing files on the side table takes a split-second longer than reaching for something on the desk itself, but it gives you enough space to handle massive workloads that the desk alone could never accommodate—such as high-resolution camera frames, graphical display buffers, or local AI models.

In short: **Choose ESP32-WROOM** for cost-effective, standard IoT sensor nodes and Wi-Fi/Bluetooth applications whose data structures easily fit within internal SRAM, and **choose ESP32-WROVER** whenever your firmware handles large memory payloads—such as JPEG image buffering, full-screen color display rendering, or machine learning model storage—that exceed the SoC's internal memory limit.

While WROOM and WROVER are the staple modules for general-purpose ESP32 development, the broader IoT ecosystem utilizes a wide array of specialized modules. From ultra-compact **PICO** packages for wearables and single-core **SOLO** chips for cost-sensitive automation, to modern **RISC-V-based C-series and H-series modules** supporting Wi-Fi 6, Zigbee, and Thread, hardware developers choose their module based entirely on the physical footprint, processing power, and wireless standards required by the application.

1.1.4 Deep Dive: Layer 3 — The Development Board (DevKit)

A **development board (DevKit)** — such as the ESP32 DevKit V1 or NodeMCU-32S — is the board a beginner actually purchases and plugs into a laptop. It takes the PCB module from Layer 2 and adds every remaining piece needed to bridge that module to an everyday USB port and a breadboard, none of which is present on the module itself.

Three specific additions make this bridge possible:

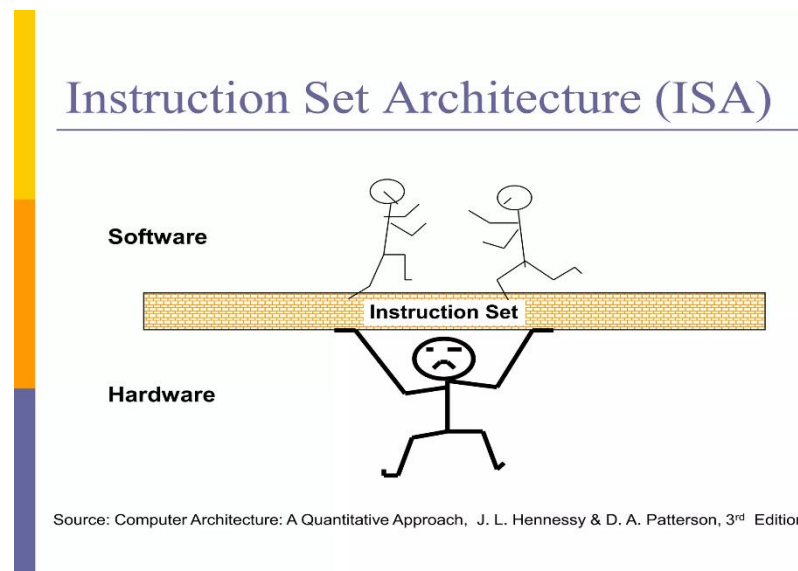
- **Power Regulation:** a standard USB port supplies 5 V, but the ESP32 module's logic and radio circuitry require a steady 3.3 V. An onboard voltage regulator — a small three-pin component near the USB connector — continuously steps 5 V down to a clean, stable 3.3 V, protecting the module from damage and from voltage fluctuations.
- **USB-to-UART Bridge IC:** a laptop's USB port and the ESP32's native serial UART interface do not speak the same electrical protocol directly. A dedicated bridge chip — commonly the CP2102 or the CH340 — sits between them, translating standard USB packets into the

simple, chip-native UART signal the ESP32 actually understands, and back again. This is what lets a program on your computer 'talk' to the chip over a normal USB cable.

- **Prototyping Interface:** rows of 2.54 mm-spaced pin headers — the same spacing used by standard breadboards and jumper wires — expose every useful signal from the module, so a beginner can wire up an LED or a sensor without any soldering. Two small push-buttons, labelled BOOT and RESET, are also included: RESET restarts the chip's program from the beginning, while BOOT (held during a reset) forces the chip into a special mode ready to receive new firmware from the computer.

1.1.5 Processor Instruction Set Architectures (ISA)

An **Instruction Set Architecture (ISA)** is the fixed vocabulary of binary commands — opcodes — that a CPU core is physically built to understand. Every line of C, Python, or any other language a developer writes is ultimately compiled down into a sequence of these opcodes before the chip can run it.



ANALOGY

Think of an ISA as a **language dictionary** built directly into a person's brain.

Two people (CPUs) can be equally smart, but if one was raised understanding only English dictionary rules and the other only French dictionary rules, a letter written in English will make no sense to the French speaker unless it is completely translated first.

Choosing an ISA means choosing the **native language** your compiled software must speak to run on that specific chip. If you try to run a program compiled for an Xtensa-based ESP32 directly on a RISC-V-based ESP32-C3 without recompiling it, the chip won't understand a single instruction.

Aspect	Tensilica Xtensa (proprietary)	RISC-V (open-source)
Used in	Classic ESP32, ESP32-S2, ESP32-S3	ESP32-C3, ESP32-P4
Licensing	Closed, licensed from Cadence/Tensilica	Freely licensed, community-governed, no royalties
Toolchain	Vendor-supplied compiler support	Mature open-source GCC and LLVM support
Customization	Limited — controlled by the IP owner	Vendors can add custom instruction extensions

Why the Semiconductor Industry is Shifting to RISC-V

The broader semiconductor industry- including Espressif-is steadily moving away from proprietary ISAs like Xtensa in favor of **RISC-V**. This transition is driven by three major engineering and economic advantages:

1. **Eliminating Royalties & Reducing Cost:** Proprietary ISAs require chip manufacturers to pay licensing fees and royalties for every single silicon chip produced. RISC-V is completely open-source, eliminating these per-chip fees and lowering production costs for IoT devices.
2. **Universal Open-Source Toolchains:** Instead of depending on a single chip vendor to supply and maintain compiler software (like GCC/LLVM), developers working on RISC-V have access to a massive, collaborative global software ecosystem.
3. **Hardware Customization Freedom:** With RISC-V, engineers are not locked into a rigid instruction set. They can design **custom instruction-set extensions** tailored specifically to intensive workloads—such as math accelerations for local AI or encryption—without needing anyone's commercial permission.

1.1.6 Native USB Hardware Capabilities

Newer ESP32 variants integrate a physical USB controller directly into the main silicon die, which can reduce or entirely eliminate the need for the external USB-to-UART bridge chip found on older development boards (as described in Section 1.1.4).

Depending on the specific chip variant (such as the ESP32-S2, S3, or C3), Espressif implements **two distinct native USB capabilities** that serve fundamentally different engineering purposes:

1. USB Serial / JTAG (Development & Debugging Interface)

What it is: Dedicated embedded hardware built directly into the silicon that allows the chip to flash new firmware and print serial debug output straight over a USB cable—requiring zero external bridge ICs.

Primary Purpose: This is purely a development and debugging convenience. It handles firmware uploading and real-time serial monitor logging (`Serial.println()`) directly through the native pins, and supports hardware JTAG debugging for stepping through code line-by-line.

Limitation: It cannot interact with external USB peripherals like mice, keyboards, or storage drives.

2. USB OTG (On-The-Go — Advanced Peripheral Controller)

What it is: A versatile hardware controller that allows the microcontroller to dynamically change its operating role between a **peripheral device** and a **master controller**.

Operational Modes:

USB Device Mode: The ESP32 acts as a peripheral gadget. For instance, you can program it to plug into a laptop and emulate a USB mouse, a custom macro keyboard, or a MIDI musical instrument.

USB Host Mode: The ESP32 acts like a computer! It can supply 5V power over the USB line and control external hardware. In Host mode, the ESP32 can read data from a USB flash drive, interface with digital cameras, or talk to USB input devices.

Limitation: A basic USB Serial/JTAG port cannot perform these tasks; USB OTG requires specialized hardware routing, pins, and software drivers.

Summary Comparison: USB Serial/JTAG vs. USB OTG

Feature / Attribute	USB Serial / JTAG	USB OTG (On-The-Go)
Primary Purpose	Flashing code and monitoring serial debug logs	Acting as a USB gadget or controlling external USB peripherals
Control Role	PC acts as host; ESP32 is the target/slave	ESP32 can act dynamically as either Host or Device
Can Read a Flash Drive?	No	Yes (in Host mode)
Can Emulate a Keyboard?	No	Yes (in Device mode)
Target ESP32 Models	ESP32-C3, ESP32-S3	ESP32-S2, ESP32-S3, ESP32-P4

USB Serial / JTAG: dedicated embedded hardware that lets the chip flash new firmware and print serial debug output directly over a USB cable — with no external bridge IC required at all. This is purely a development and debugging convenience.

USB OTG (On-The-Go): hardware that lets the chip itself act as either a USB host (the role your laptop normally plays) or a USB device (the role a mouse or flash drive normally plays). In host mode, the ESP32 can talk to peripherals such as keyboards, mice, USB flash drives, or MIDI musical devices — something a simple USB Serial/JTAG interface cannot do.

WHY THIS MATTERS

USB Serial/JTAG answers the question 'how do I program and debug this chip?' USB OTG answers a completely different question: 'can this chip control other USB devices once it's running?' A board can have one, both, or neither, depending on which ESP32 variant it is built around.

1.1.7 Key Terms at a Glance

A quick-reference glossary for the terms introduced in this section:

Term	Plain-Language Definition
SoC	A single silicon chip containing a full processor and its core supporting circuitry.
MCU	A chip with CPU + RAM + storage/peripherals all on one die; needs almost nothing external.
MPU	A chip with only a CPU core on the die; needs external RAM, storage, and a PMIC.
SRAM	Fast, volatile on-chip memory used as active working space while code runs.
Flash	Non-volatile memory that stores the program itself, even with power off.
PSRAM	External volatile memory added on WROVER modules for large workloads.
GPIO	A general-purpose pin that firmware can read (input) or drive (output).
UART / SPI / I2C	Hardware controllers that manage serial communication protocols.
PMIC	A chip that regulates and distributes power to multiple voltage rails.
ISA	The fixed set of binary instructions a CPU core is built to execute.
USB OTG	USB hardware allowing a chip to act as host or device.

1.1.8 Section Recap

- Hardware is built in three nested layers: Silicon Chip → PCB Module → Development Board, each adding what the previous layer lacks.
- MCU vs. MPU is decided at the silicon level: MCUs integrate RAM/storage on-die; MPUs require external RAM, storage, and a PMIC.
- Modules exist to solve RF compliance, impedance matching, and shielding — problems best solved once, by specialists.
- DevKits exist to bridge a module to USB and a breadboard via power regulation, a USB-to-UART bridge, and header pins.

-
- The ISA (Xtensa vs. RISC-V) determines the low-level 'language' compiled firmware must speak to run on a given chip.
 - Native USB Serial/JTAG (for flashing/debugging) and USB OTG (for host/device roles) are distinct capabilities, not interchangeable.

1.2 The ESP32 Ecosystem Overview

This section introduces the ESP32 family from first principles: what it is, who makes it, how the modern product line breaks down into distinct chip variants, and why an engineer would reach for one instead of a traditional microcontroller. Sections 1.0 through 1.3 build the big-picture map; Section 1.4 then drops down a level to the fundamental embedded-hardware vocabulary used throughout the rest of this report.

1.2.0 Introduction: What Is the ESP32 and Why Use It?

Espressif Systems is a semiconductor company, founded in Shanghai in 2008, that designs low-cost System-on-Chip (SoC) devices built specifically for wireless-connected embedded products. “ESP” is simply the brand prefix Espressif uses across its entire chip catalogue — in the same way “STM” identifies a family from STMicroelectronics. “ESP32” names the specific 32-bit processor generation that succeeded Espressif’s earlier ESP8266, and it is not one fixed chip but an entire, still-growing family of variants that share a common design philosophy: combine a capable processor with a radio on a single, inexpensive piece of silicon.

In plain terms

Think of “ESP32” the way you’d think of “iPhone” — it’s a product line, not a single model. Just as there’s an iPhone with a bigger camera and one with a smaller battery, there’s an ESP32 variant with more RAM, one with no Bluetooth at all, and one built purely for high-speed video. Picking “the ESP32” for a project really means picking the right family member for the job.

The family has grown considerably since the original ESP32 launched in 2016. By 2026 the lineup spans eleven distinct System-on-Chip variants, split roughly into three lines: the original Xtensa-based chips (the classic ESP32 and the S-series), a newer generation built on the open RISC-V instruction set (the C-series and H-series), and a high-performance multimedia chip with no wireless radio at all (the P4). Each new variant has traded some capability for another — more RAM here, a faster clock there, Zigbee and Thread support instead of Bluetooth — which is precisely why Section 1.1 below surveys the individual variants before anything else in this report is discussed. This report focuses on the five variants an engineer is most likely to encounter first: the classic ESP32, the S2, the S3, the C3, and the P4.

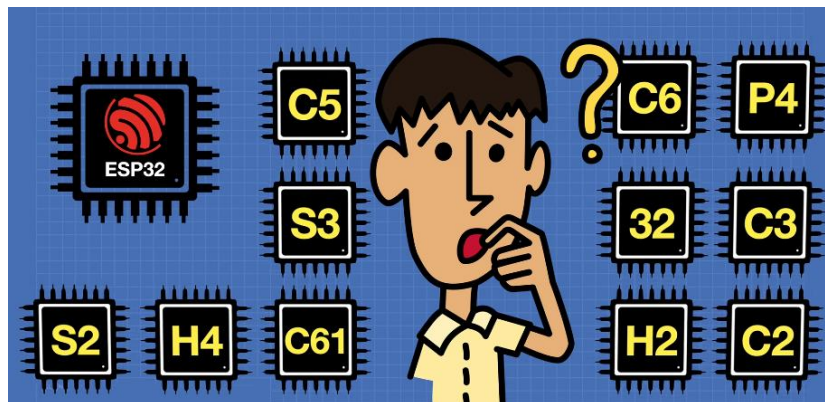
Why choose an ESP32 at all, instead of a traditional 8-bit microcontroller such as the Microchip ATmega328P (the chip at the heart of the classic Arduino Uno) or a 32-bit ARM Cortex-M chip such as an STMicroelectronics STM32? The short answer is integration and raw capability at a very low price. The longer answer breaks down into five concrete

advantages:

- **Integrated wireless radios.** Every mainstream ESP32 variant except the P4 includes Wi-Fi and Bluetooth Low Energy directly on the die. An ATmega328P or STM32 project that needs wireless connectivity must add a separate radio module (such as an ESP8266, an nRF24L01, or a Wi-Fi shield), wire it up, and write a driver for it — the ESP32 simply skips that entire step.
- **High clock speeds.** Where an ATmega328P tops out at 16 MHz, most ESP32 cores run at 160 MHz to 240 MHz, and the newer P4 reaches 400 MHz — an order of magnitude more headroom for signal processing, encryption, or driving a display.
- **Generous SRAM and Flash.** An ATmega328P offers 2 KB of SRAM and 32 KB of flash. Even the most modest ESP32 variant offers well over 100 times that SRAM, which is what makes a full TCP/IP network stack, a TLS encryption library, and application code all fit comfortably in memory at once.
- **Rich, modern peripherals.** Capacitive touch inputs, hardware I2S audio, a built-in DAC (on some variants), CAN/TWI controllers, and — on the S-series and P4 — native USB and camera interfaces come standard, reducing the number of external support chips a design needs.
- **An open, modern software ecosystem.** The ESP-IDF (Espressif IoT Development Framework) is open source and actively maintained, and the chip is also fully supported by the Arduino IDE, MicroPython, and ESPHome — giving a beginner the same on-ramp a professional team uses for production firmware.

None of this makes the ESP32 a universal replacement for an ATmega328P or STM32; Section 1.2.3 returns to this comparison in more depth and is honest about where the trade-offs cut the other way.

1.2.1 Board Survey & Core ESP32 Variants



The remainder of this report works primarily with five core ESP32 variants. Each one represents a different point on the trade-off curve between processing power, wireless capability, cost, and USB support, so it is worth knowing each by name before comparing them side by side in Section 1.2.2.

- **ESP32 (Classic / WROOM / WROVER).** The original member of the family: a dual-core Xtensa LX6 processor with both Bluetooth Classic and BLE. “WROOM” and “WROVER” are the names of the pre-built modules built around this die — WROVER adds external PSRAM for memory-hungry applications. It remains the general-purpose workhorse of the family and the chip most existing tutorials are written for.
- **ESP32-S2.** A single-core Xtensa LX7 chip that drops Bluetooth entirely in exchange for a full-featured USB On-The-Go (OTG) port and an onboard 8-bit DAC. It suits projects that need native USB — such as a custom USB keyboard or a USB-to-something bridge — but have no need for Bluetooth.
- **ESP32-S3.** A dual-core Xtensa LX7 chip that pairs Wi-Fi 4 and BLE 5 with the same native USB OTG support as the S2, plus AI vector-processing instructions (Espressif calls these Processor Instruction Extensions, or PIE) built into its cores. Those extensions make it the family’s natural choice for TinyML and edge-AI workloads such as wake-word detection or lightweight image classification, and it is currently the most widely used ESP32 variant overall.
- **ESP32-C3.** A single-core, low-cost chip built on the open RISC-V instruction set rather than Xtensa. It supports Wi-Fi and BLE and is positioned as the modern, more secure successor to the older ESP8266, making it a popular choice for simple, price-sensitive sensor nodes.
- **ESP32-P4.** A high-performance dual-core RISC-V chip clocked at 400 MHz, built for demanding human-machine interface (HMI) and multimedia workloads — driving displays, decoding camera feeds, and running graphics-heavy user interfaces. Unlike every other variant in this survey, the P4 has no wireless radio on the die at all; designs that need Wi-Fi or Bluetooth typically pair it with a companion chip such as an ESP32-C6.

1.2.2 Ecosystem Comparison Matrix

The table below places the five core variants side by side across the specifications that matter most when choosing a chip for a new design.

Variant Name	Core Architecture	Clock Speed	Wi-Fi / BLE Specs	Native USB Support(JTAG/OTG)	Ideal Use Cases
ESP32 (Classic / WROOM / WROVER)	Xtensa LX6, dual-core	Up to 240 MHz	Wi-Fi 4 (2.4 GHz); Bluetooth Classic + BLE 4.2	JTAG	General-purpose IoT, legacy Bluetooth-Classic projects, learning/tutorials
ESP32-S2	Xtensa LX7, single-core	Up to 240 MHz	Wi-Fi 4 (2.4 GHz); no Bluetooth/BLE	Yes — full USB OTG	USB HID devices, DAC-based audio/analog output, no-Bluetooth designs
ESP32-S3	Xtensa LX7, dual-core	Up to 240 MHz	Wi-Fi 4 (2.4 GHz); BLE 5.0	Yes — full USB OTG	TinyML / edge AI, camera + display projects, general-purpose prototyping
ESP32-C3	RISC-V, single-core	Up to 160 MHz	Wi-Fi 4 (2.4 GHz); BLE 5.0	JTAG	Low-cost sensor nodes, ESP8266 replacement, budget IoT deployments

Variant Name	Core Architecture	Clock Speed	Wi-Fi / BLE Specs	Native USB Support(JTAG/OTG)	Ideal Use Cases
ESP32-P4	RISC-V, dual-core	Up to 400 MHz	None on-die (requires companion Wi-Fi/BLE chip)	Yes — USB 2.0 OTG	High-performance HMI, displays, camera/video pipelines, edge multimedia

A recurring pattern is worth calling out explicitly: native USB support and Bluetooth are largely mutually exclusive on the Xtensa-based chips (the S2 and S3 trade Bluetooth Classic away, and the classic ESP32/WROOM has no native USB at all), while the RISC-V-based chips trade raw clock speed for lower cost and power. No single variant wins on every axis, which is exactly why Section 1.1’s survey and this matrix exist — the right chip is a function of the project’s requirements, not a single “best” ESP32.

1.2.3 Pros & Cons: ESP32 vs. Traditional Microcontrollers

Section 1.1.0 introduced the case for the ESP32; this section weighs that case more formally against traditional microcontrollers such as the ATmega328P and the STM32 family, covering both the structural advantages the ESP32 brings and the real trade-offs it asks an engineer to accept.

Structural Advantages:

- **Integrated wireless.** Wi-Fi and Bluetooth are built into the silicon on most variants, removing an entire category of external radio modules, antennas, and driver code that an ATmega328P or STM32 design would otherwise need.
- **High compute capability.** Clock speeds of 160–240 MHz (400 MHz on the P4), paired with large SRAM and flash budgets, let the ESP32 run workloads — TLS-encrypted networking, JSON parsing, on-device machine learning — that would be impractical on an 8-bit MCU and would demand a more expensive external MPU on a traditional design.
- **True multitasking via FreeRTOS.** The ESP32 toolchain ships with FreeRTOS built in and running by default, giving even beginner projects real preemptive multitasking (e.g., a Wi-Fi task running independently of a sensor-polling loop) without the engineer having to write a scheduler from scratch, as is common on bare-metal ATmega328P projects.
- **Low cost for the capability delivered.** A bare ESP32 module frequently costs less than a Wi-Fi shield alone would cost to add to a traditional MCU, making the ESP32 difficult to beat on a dollars-per-capability basis for any project that needs connectivity.

Trade-offs and Disadvantages:

- **Higher active power consumption.** An ATmega328P can idle in a deep-sleep mode drawing single-digit microamps for months on a coin cell; an ESP32 with its radio active draws tens to hundreds of milliamperes. Battery-powered designs

with extreme runtime requirements often still favor an ultra-low-power 8-bit or Cortex-M0 MCU, or rely carefully on the ESP32's own deep-sleep modes to close the gap.

- **A steeper learning curve.** FreeRTOS, Wi-Fi/BLE stacks, and a much larger set of peripherals mean there is simply more surface area to learn than on a simple 8-bit MCU, and concurrency bugs (race conditions between tasks) are a class of problem that an ATmega328P programmer rarely has to think about at all.
- **Real-time determinism is weaker.** A traditional Cortex-M-based MCU such as the STM32 running bare-metal or a hard-real-time OS can guarantee tighter, more predictable interrupt latency than a general-purpose FreeRTOS task on the ESP32, which matters for applications like precision motor control.
- **Board and pin-count overhead.** The extra RF circuitry, antenna, and larger die mean an ESP32 module is physically larger than a bare ATmega328P, which can matter in extremely space-constrained designs.

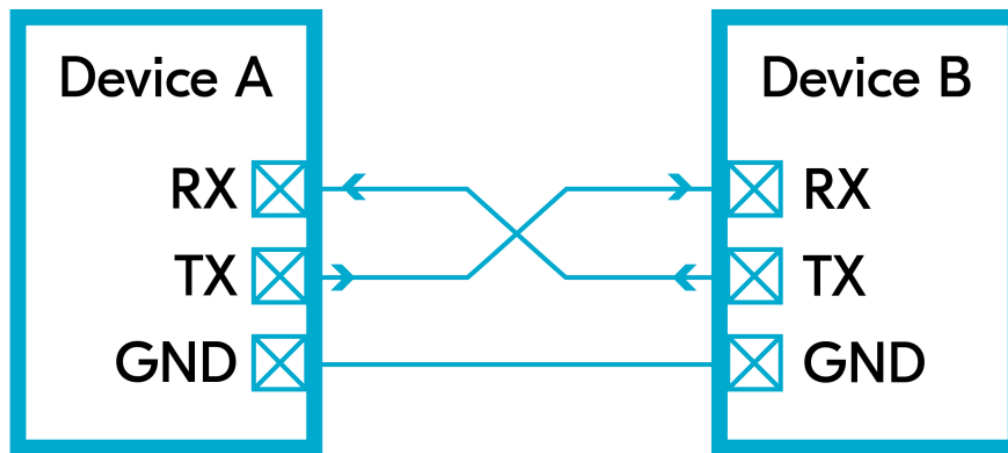
Taken together, the ESP32 is best understood not as a strictly “better” chip than the ATmega328P or STM32, but as a different tool optimized for connected, compute-heavier applications — the traditional microcontrollers remain the right choice when ultra-low power or hard real-time guarantees are the top priority.

Section 2: Communication Protocols Deep-Dive

The previous section established what an embedded system is and how its hardware is packaged. None of that hardware is useful in isolation — a sensor node has to hand its readings to a microcontroller, and a microcontroller often has to hand results to a display, a memory chip, or another board entirely. This section covers the four wired serial protocols that make those handoffs possible: UART, I²C, SPI, and RS-485. Wireless protocols such as Wi-Fi, Bluetooth, and Zigbee are covered separately elsewhere in this report and are intentionally out of scope here. As in Section 1, every acronym is spelled out on first use, and every protocol opens with an everyday analogy before the formal engineering definition, so that a reader with no prior serial-communication background can follow the whole section end to end.

Each protocol below is analyzed against the same four metrics, so they can be compared apples-to-apples: (1) Topology & Wire Count, (2) Typical Speeds & Maximum Practical Distance, (3) Bus Arbitration & Handling Multiple Devices, and (4) Pros, Cons, and Primary Industrial/Embedded Application Examples. A consolidated comparison table closes out the section.

2.1 UART (Universal Asynchronous Receiver-Transmitter)



Picture two people standing far enough apart that they can't watch each other's lips, agreeing beforehand on exactly how fast they'll speak, then taking turns talking without any shared metronome keeping them in step — each side simply trusts the agreed-upon pace. That is precisely how a UART behaves. UART stands for Universal Asynchronous Receiver-Transmitter, and the word "asynchronous" is the key: there is no shared clock wire at all. Instead, both sides are hard-configured to the same speed (the baud rate) in advance, and every packet of data is bookended with a start bit and a stop bit so the receiving side knows exactly when a new byte begins and ends.

In plain terms

A UART is a walkie-talkie conversation between exactly two devices. There's no group channel and no names spoken — just two parties, a pre-agreed talking speed, and clear "I'm starting" / "I'm done" signals bookending every word.

1. Topology & Wire Count

UART has no controller/target (master/slave) hierarchy at all — it is a point-to-point link between exactly two devices, and both sides are electrical peers. The wiring is famously minimal: a TX (transmit) wire from device A into the RX (receive) wire of device B, a TX wire from device B into the RX wire of device A, and a shared ground (GND) reference. That's 2 signal wires plus 1 ground wire, cross-connected (TX-to-RX on each side) rather than wired straight across.

2. Typical Speeds & Maximum Practical Distance

Common baud rates run from 9,600 bps (a long-standing default for simple modules and debug consoles) up to 115,200 bps for everyday use, with dedicated hardware UART peripherals on chips like the ESP32 capable of a few Mbps under the right conditions. Because the signal is single-ended (measured against ground, not as a differential pair) and unbuffered, practical cable runs are short: on the order of a couple of metres at higher baud rates, stretching to a few tens of metres at low baud rates over well-shielded, twisted-pair-style cabling. Longer runs are usually handed off to a signal-level converter such as RS-232 or RS-485 rather than pushed over raw UART wiring.

3. Bus Arbitration & Handling Multiple Devices

There is no arbitration mechanism because there is no bus to arbitrate — UART is inherently a one-to-one link, and neither side has an address. Connecting a third device to the same pair of wires does not work without extra hardware, such as a UART multiplexer or a separate hardware UART peripheral per connected device. This is precisely why UART is well suited to simple, fixed one-to-one links (a GPS module talking to a single microcontroller, for instance) but is the wrong choice the moment a design needs one controller to talk to many peripherals on shared wiring.

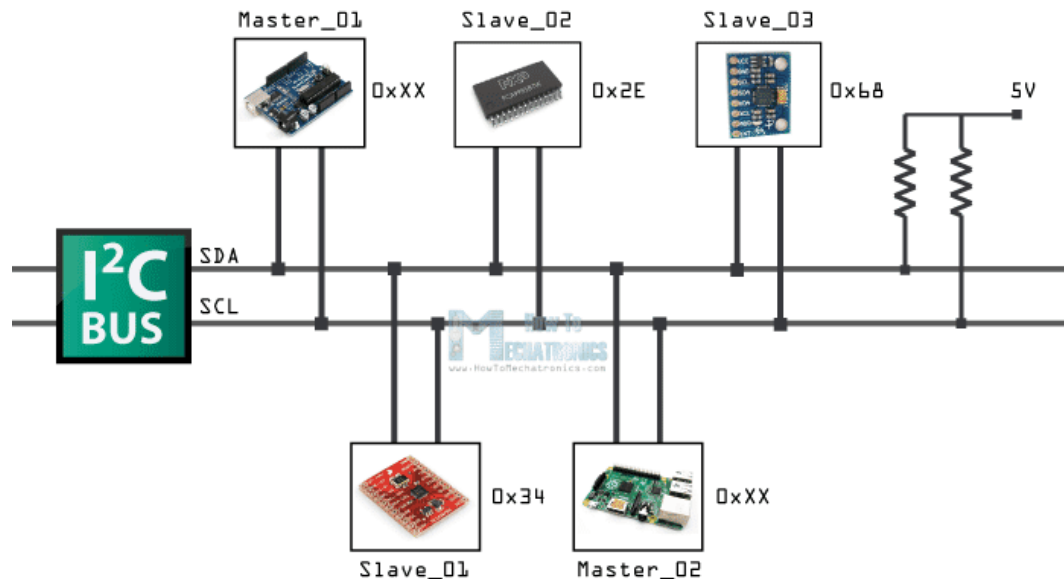
4. Pros, Cons, and Primary Applications

Pros	Cons
• Extremely simple to implement in both hardware and software • Needs only two signal wires plus ground • Universally supported — nearly every microcontroller has at least one hardware UART • Low cost and low pin count	• Strictly point-to-point; cannot natively address multiple devices on shared wiring • Both sides must be configured to the exact same baud rate, or communication fails silently • No built-in error checking beyond an optional single parity bit • Practical range is short without a signal-level converter

Primary Industrial / Embedded Application Examples

- The USB-to-UART bridge chip (CP2102 / CH340) used to flash and debug ESP32 DevKits, as covered in Section 1.1.4
- GPS receiver modules reporting position data to a microcontroller
- Point-to-point links to Bluetooth serial modules and other simple sensor breakout boards

2.2 I²C (Inter-Integrated Circuit)



Imagine an old office intercom system where dozens of desk extensions all share the same two wires. To reach a specific colleague, you don't shout into the line — you first announce their extension number, and only that one person picks up and answers while everyone else stays silent. That's the essence of I²C (pronounced "I-squared-C," for Inter-Integrated Circuit). I²C was developed by Philips Semiconductor in the early 1980s as a way to let a single controller chip talk to many peripheral chips — sensors, memory, real-time clocks — over just two shared wires, using an address system rather than dedicated wiring per device.

In plain terms

I²C is a party line with named extensions. Every device shares the same two wires, but each device has its own address, so a message intended for the temperature sensor is quietly ignored by the display and the clock chip sitting on the very same wires.

1. Topology & Wire Count

I²C uses a controller/target relationship (traditionally called master/slave): one controller device — usually the microcontroller — initiates every transaction, though the specification also supports multiple controllers sharing the same bus. The wiring is just 2 signal wires: SDA (Serial Data), which carries the actual bits in both directions, and SCL (Serial Clock), which the controller drives to pace

every bit transfer, plus a shared ground. Both lines are open-drain and rely on pull-up resistors to rest at a logic-high level when idle.

2. Typical Speeds & Maximum Practical Distance

I²C defines several standard speed grades: Standard Mode (100 kbps), Fast Mode (400 kbps, the most commonly used speed in practice), Fast Mode Plus (1 Mbps), and High-Speed Mode (3.4 Mbps). Because the bus relies on pull-up resistors charging shared wiring that behaves like a small capacitor, distance is limited — well under a metre in a typical onboard design, extending to a few metres only with careful pull-up tuning or dedicated I²C bus-extender/buffer chips.

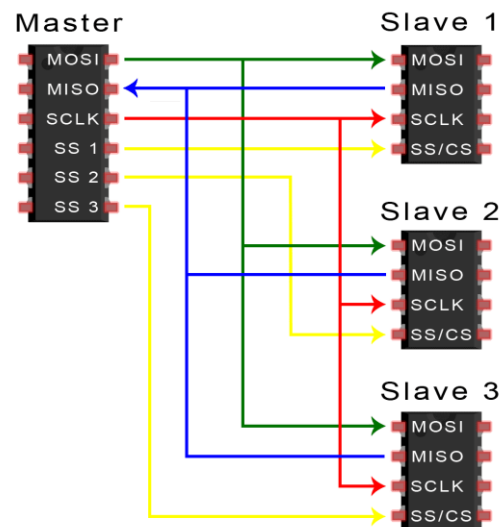
3. Bus Arbitration & Handling Multiple Devices

Every device on an I²C bus is identified by a 7-bit (or, less commonly, 10-bit) address, so a single controller can address dozens of different peripherals over the same two wires without any additional wiring per device — the controller simply states the target's address at the start of every transaction, and only that device responds. When more than one controller shares a bus, I²C also defines a collision-detection arbitration scheme, where a controller that notices another controller driving the bus differently than expected quietly backs off. A related mechanism called clock stretching lets a slower target device hold the clock line low to buy itself more time mid-transaction.

4. Pros, Cons, and Primary Applications

Pros	Cons
• Only 2 wires needed no matter how many devices are on the bus • Built-in addressing means no extra select wires per device • Supports multiple controllers on the same bus • Very common on sensor and peripheral breakout boards	• Slower than SPI at comparable pin counts • Bus capacitance limits practical distance and device count • Software/protocol implementation is more involved than SPI • A single misbehaving device can "hang" the shared bus
Primary Industrial / Embedded Application Examples • Onboard sensor hubs — IMUs, temperature/humidity sensors, ambient light sensors • Small OLED/LCD display modules • Real-time clock (RTC) chips and EEPROM memory	

2.3 SPI (Serial Peripheral Interface)



Picture a strict teacher running a classroom drill: a metronome keeps a fixed beat, and the teacher taps exactly one student on the shoulder at a time to signal "you're up" — that student, and only that student, speaks and listens simultaneously until the teacher moves on. That's SPI. The Serial Peripheral Interface was developed by Motorola in the late 1980s as a synchronous, full-duplex protocol — full-duplex meaning data can flow in both directions at the very same instant, unlike UART or I²C, which mostly take turns. A single controller drives the clock and individually selects which target device is "live" at any given moment.

In plain terms

SPI is a teacher (the controller) with a shared chalkboard and clock, but a private tap on the shoulder — a dedicated select wire — for every single student. Whoever gets tapped is the only one allowed to answer, and they can talk and listen at exactly the same time.

1. Topology & Wire Count

SPI uses a strict controller/target relationship: one controller, one or more targets, and the controller alone drives the clock and initiates every transaction. Three wires are shared by every device on the bus — SCLK (Serial Clock, driven by the controller), MOSI (Controller/Master Out, Target/Slave In), and MISO (Controller/Master In, Target/Slave Out) — plus one dedicated Chip Select (CS, also called Slave Select or SS) wire that must be run individually to every single target device. A design with one target therefore needs 4 wires total; a design with four targets needs 3 shared wires plus 4 individual CS wires, for 7 wires in total.

2. Typical Speeds & Maximum Practical Distance

SPI is the fastest of the four protocols covered in this section, commonly running at several MHz and often into the tens of MHz — modern microcontrollers such as the ESP32 can drive many SPI peripherals well past 40 MHz. That speed comes at the cost of range: SPI is intended for short, low-

noise runs on the same PCB or over a short ribbon cable, typically well under 30 cm (about 1 foot), since the unshielded, single-ended wiring becomes unreliable at high clock speeds over any real distance.

3. Bus Arbitration & Handling Multiple Devices

SPI has no arbitration scheme at all, because it doesn't need one — the controller has complete, exclusive authority over the bus at every moment, so there is never any ambiguity about who is allowed to transmit. Multiple targets are handled through the dedicated CS lines described above: pulling a specific target's CS line low wakes that device up and puts every other target on the bus to sleep for the duration of the transaction. Where CS pins are scarce, some SPI devices (such as certain shift registers and LED drivers) instead support a daisy-chain arrangement, where a single CS line activates a whole chain and data shifts through one device into the next in sequence.

4. Pros, Cons, and Primary Applications

Pros	Cons
• Highest raw throughput of the four protocols in this section • True full-duplex — send and receive happen simultaneously • Simple receiving hardware (essentially a shift register) • No arbitration overhead — the controller is always in charge	• Needs a dedicated CS wire per target, which quickly consumes GPIO pins • No official, universally enforced standard — implementations vary between vendors • Not designed for distances beyond a single board • Single point of failure: if the controller stops, the whole bus stops
Primary Industrial / Embedded Application Examples • SD/microSD card interfaces • TFT and OLED display modules • External flash memory and ADC/DAC chips	

2.4 RS-485 (Industrial Differential Serial Communication)



Imagine two people carrying a long, wobbly plank through a crowded, noisy factory floor. Instead of one person shouting instructions over the noise (which gets lost), they use a rehearsed hand signal system where what matters is the difference between their two hand positions, not either hand alone — factory noise that jostles both hands equally cancels itself out, because it's the difference that carries the message. That's the core trick behind RS-485: rather than measuring a signal against a shared ground wire (as UART, I²C, and SPI all do), RS-485 sends every bit as the voltage difference between two wires, A and B. Any electrical noise that hits both wires equally — which is most noise in a real factory — cancels out at the receiver, which is exactly why RS-485 tolerates the electrically noisy environments common on factory floors, in a way the other three protocols in this section cannot.

In plain terms

RS-485 doesn't ask "what's the voltage on this one wire?" — it asks "which of these two wires is higher than the other, right now?" Because factory noise tends to hit both wires the same way, that question keeps giving a clean answer even next to running motors and welding equipment, over distances measured in kilometres rather than centimetres.

1. Topology & Wire Count

RS-485 itself only defines the electrical signaling layer, not a full communication protocol — it's the wiring and voltage rules that a higher-layer protocol, most often Modbus RTU, runs on top of. Physically, it uses just 2 signal wires (the differential A/B pair), plus an optional common ground reference for very long runs, wired as a multi-drop bus where every device taps into the same twisted pair. The standard supports up to 32 standard-load devices on one bus, extendable into the

hundreds with modern reduced-load transceiver chips. Whether the bus behaves as controller/target or peer-to-peer is decided entirely by the higher-layer protocol riding on top — Modbus RTU, for example, imposes a single-controller/multiple-target structure using device ID numbers.

2. Typical Speeds & Maximum Practical Distance

RS-485 trades speed for distance, and the two move in opposite directions: at its maximum practical cable length of roughly 1,200 metres (about 4,000 feet), speeds are limited to around 100 kbps, while over short runs on the order of tens of metres, RS-485 links can reach up to about 10 Mbps. This distance/speed trade-off is precisely what makes RS-485 the default choice for spreading a single control network across an entire factory floor or building, somewhere none of the other three protocols in this section could reach.

3. Bus Arbitration & Handling Multiple Devices

RS-485 itself defines no arbitration or addressing scheme — that responsibility is pushed up to whichever higher-layer protocol is running over it. In the common case of Modbus RTU, a single controller polls each target device in turn by its numeric device ID, and because RS-485 is fundamentally half-duplex (only one device may drive the differential pair at a time), every device on the bus must stay silent unless it is the one currently being addressed. Collisions are avoided procedurally, by the polling discipline of the higher-layer protocol, rather than electrically, by the RS-485 standard itself.

4. Pros, Cons, and Primary Applications

Pros	Cons
• Very long cable runs — up to roughly 1,200 m • Strong noise immunity from differential signaling, ideal for electrically noisy environments • Multi-drop bus supports up to 32 (or more) devices on one pair of wires • Low per-node hardware cost	• Typically half-duplex — devices must take turns, adding turn-around delay • No addressing or framing of its own; needs a higher-layer protocol such as Modbus RTU • Requires a dedicated RS-485 transceiver IC in addition to a UART • Speed and distance trade directly against each other
Primary Industrial / Embedded Application Examples • Modbus RTU networks in industrial automation and process control • Building automation systems (e.g., BACnet MS/TP) • Variable-frequency motor drives and solar charge-controller networks	

2.5 Summary Comparison Table

The table below places all four protocols side by side across the metrics that matter most when choosing a wired link for a new design. As a general rule of thumb: reach for UART for a single simple point-to-point link, I²C when many low-speed onboard peripherals need to share the fewest possible pins, SPI when raw on-board speed matters more than pin count, and RS-485 the moment a signal has to survive real industrial distance and electrical noise.

Protocol	Wire Count	Topology	Max Distance	Speed	Primary Use Case
UART	2 signal wires (TX, RX) + GND, cross-connected	Point-to-point (no controller/target roles — two peers)	~2 m at high baud rates; tens of metres at low baud rates (TTL levels)	Typically 9,600 bps – 115,200 bps; hardware peripherals up to a few Mbps	Debug consoles, GPS/Bluetooth modules, USB-to-serial bridges
I ² C	2 signal wires (SDA, SCL) + GND	Multi-controller / multi-target bus, 7- or 10-bit addressing	Well under 1 m typically; a few metres with strong pull-ups or bus buffers	100 kbps (Standard) to 3.4 Mbps (High-Speed); 400 kbps (Fast Mode) most common	Onboard sensors (IMU, temp/humidity), RTC chips, small OLED displays, EEPROMs
SPI	4 shared wires (SCLK, MOSI, MISO) + 1 dedicated CS per target	Single controller, multiple targets selected individually via CS lines	Well under 0.3 m (same PCB or short ribbon cable)	Several MHz up to tens of MHz (often 10–80 MHz on modern MCUs)	SD cards, TFT/OLED displays, flash memory, ADC/DAC chips
RS-485	2 wires (differential pair A/B) + optional common GND	Multi-drop bus, up to 32 standard-load devices (more with extended-load transceivers); roles defined by the higher-layer protocol	Up to ~1,200 m (about 4,000 ft) at low speed; distance shrinks as speed rises	100 kbps at maximum distance, up to ~10 Mbps at short range	Modbus RTU industrial automation, building automation (BACnet MSTP), motor drives

Section Recap: UART is a simple two-wire, point-to-point link with no addressing. I²C shares two wires among many addressed devices at moderate speed and short range. SPI trades extra select wires for the highest on-board speed and true full-duplex transfer. RS-485 sacrifices raw speed and duplex operation for long-distance reach and strong noise immunity, making it the standard choice once a wired link

Section 3: Power Electronics & Sizing Fundamentals

Sections 1 and 2 covered the digital side of an embedded system — the processor, its memory, and how it talks to other chips over short-range serial buses. None of that digital logic can run without a clean, correctly sized supply of electrical power, and the moment a project adds a real-world actuator — a motor, a solenoid, a heater — power stops being an afterthought and becomes a design problem in its own right. This section builds that foundation from first principles: Section 3.1 explains the three families of power electronics that convert energy from one electrical form into another, and Section 3.2 walks through a complete, professional worked example of sizing a DC power supply for a real project in this program.

3.1 Converters vs. Inverters (Power Electronics Fundamentals)

Electrical power comes in only two basic ‘flavors’: Alternating Current (AC), which is what a wall outlet supplies and periodically reverses direction (50–60 times a second), and Direct Current (DC), which flows in one direction only and is what every microcontroller, sensor, and battery actually needs. A **power converter** is any circuit whose job is to translate electrical energy from one of these forms, or one voltage level, into another that a downstream device can safely use.

In plain terms

Think of power converters as translators at an international airport. AC and DC are two different ‘languages’ that electrical energy can ‘speak.’ A rectifier translates AC into DC, an inverter translates DC back into AC, and a DC/DC converter is a translator who stays within one language but adjusts the volume — stepping a DC voltage up or down to match what the listener (the load) needs.

All three converter families below are built from the same handful of building blocks — diodes, transistors used as switches, transformers, capacitors, and inductors — arranged differently depending on which direction the energy needs to flow. The three families are examined in the order a beginner is most likely to meet them: converting mains AC down to usable DC, converting one DC voltage into another DC voltage, and finally converting DC back into AC.

3.1.1 AC/DC Rectifiers (AC-to-DC Conversion)

An AC/DC converter, more commonly called a **rectifier**, is the circuit that takes the alternating voltage from a wall outlet and turns it into the steady one-directional voltage that electronics actually run on. Every plug-in phone charger, laptop ‘brick,’ and benchtop power supply contains a rectifier stage of exactly this kind.

In plain terms

AC is like a crowd of people shuffling back and forth across a hallway — lots of motion, but no one actually goes anywhere. A rectifier is a set of one-way turnstiles placed across that hallway: no matter which way a person was originally shuffling, the turnstiles only let them exit in one direction. The result is a steady one-way stream instead of a back-and-forth crowd — exactly what DC is.

A complete AC/DC rectifier stage is built from four cooperating stages, each solving a specific problem left behind by the stage before it:

- **Transformer:** steps the incoming mains voltage (e.g., 120 V or 230 V AC) up or down to a safer, more useful AC voltage, and — just as importantly — electrically isolates the low-voltage electronics from the dangerous mains wiring.
- **Bridge Rectifier (four diodes):** a diode is an electronic one-way valve for current. Four diodes wired in a bridge arrangement redirect both the positive and negative halves of the incoming AC wave so that current always flows out in the same direction, producing a bumpy, pulsing DC voltage.
- **Smoothing (Filter) Capacitor:** a capacitor stores charge like a small reservoir. Placed right after the bridge rectifier, it charges up on each voltage pulse and discharges slightly in the gaps between pulses, flattening the bumpy output into a much smoother, more usable DC voltage — a process called filtering ripple.
- **Voltage Regulator:** a final active stage that trims the smoothed-but-still-imperfect DC down to one exact, stable voltage (such as a clean 5 V), holding that output steady even as the incoming mains voltage or the load's current draw fluctuates.

Because it depends on a bulky, mains-frequency transformer and passes current through diodes only during brief portions of each AC cycle, this classic ‘linear’ rectifier topology is simple and low-noise but comparatively heavy and inefficient — the DC/DC switch-mode converters in the next subsection solve exactly that inefficiency problem for the voltage-changing part of the job.

3.1.2 DC/DC Converters — Buck (Step-Down) and Boost (Step-Up)

A DC/DC converter takes a DC voltage that is already available (from a battery, or from the rectifier stage above) and changes it to a different DC voltage — without ever converting back through AC. Rather than wasting energy as heat the way a simple resistive voltage divider would, a **switch-mode** DC/DC converter turns a transistor on and off thousands to millions of times per second, and uses an inductor and capacitor to smooth those rapid pulses into a new, steady output voltage. This switching approach is what lets modern DC/DC converters routinely exceed 90% efficiency.

In plain terms

Picture someone rapidly opening and closing a water valve to fill a bucket to a precise level, instead of leaving the tap fully open and draining the excess down a side pipe (which is what a wasteful linear regulator effectively does). The faster and more precisely the valve is pulsed open versus closed — its ‘duty cycle’ — the more precisely the final water level (output voltage) can be controlled, with almost nothing wasted along the way.

The ratio of ‘on’ time to total switching-cycle time is called the **duty cycle**, and it is the single control knob that determines the output voltage. The two most common DC/DC topologies differ only in how the switch, inductor, and diode are arranged around that control knob:

Aspect	Buck Converter (Step-Down)	Boost Converter (Step-Up)
Function	Output voltage is lower than the input voltage	Output voltage is higher than the input voltage

Typical use	Stepping a 12 V supply down to 5 V or 3.3 V logic rails	Stepping a 3.7 V single-cell battery up to 5 V USB output
Core parts	Switching transistor (MOSFET), inductor, diode, output capacitor	Switching transistor (MOSFET), inductor, diode, output capacitor — rearranged
Duty cycle rule	Duty cycle < 50% typically — more ‘off’ time trims the output down	Duty cycle > 50% typically — more ‘on’ time charges the inductor higher
ESP32 relevance	Most USB-powered 5 V → 3.3 V regulators on ESP32 DevKits use a buck (or linear) regulator	Used when a single-cell LiPo battery (3.0–4.2 V) must supply a stable 5 V rail

In both topologies, the inductor is the key energy-storage element: it stores energy in a magnetic field while the switch is on, then releases that stored energy to the output when the switch turns off, and the output capacitor smooths the result into a steady DC voltage — the same filtering role the smoothing capacitor played in the rectifier stage above, just running at a far higher switching frequency.

3.1.3 DC/AC Inverters (DC-to-AC Conversion)

A DC/AC converter, or **inverter**, runs the rectifier concept in reverse: it takes steady DC — typically from a battery or a solar panel — and reconstructs an alternating output that downstream AC equipment can use. Car power outlets that run a laptop charger, solar inverters, and Uninterruptible Power Supply (UPS) units all rely on this circuit.

In plain terms

If a rectifier is a set of one-way turnstiles that force a crowd into single-file, an inverter is the opposite trick: four gates arranged so that opening one diagonal pair sends the crowd one way, and opening the other diagonal pair sends it back the other way. Switch between the two pairs fast enough, on a repeating schedule, and the crowd's overall motion looks exactly like the back-and-forth shuffle of AC — even though every individual push started out as pure DC.

The circuit that performs this diagonal switching trick is called an **H-Bridge**, named for the letter ‘H’ shape its four transistor switches form around the load. An oscillator or PWM controller drives the four transistors in two alternating diagonal pairs:

- **Pair 1 ON (top-left + bottom-right):** current flows through the load in the forward direction.
- **Pair 2 ON (top-right + bottom-left):** current flows through the same load in the reverse direction.
- **Alternating rapidly between the two pairs:** produces a current through the load that reverses direction on a regular schedule — the defining property of AC — with a step-up transformer often added afterward to reach the higher voltages AC appliances expect.

This same H-Bridge arrangement is not limited to producing mains-style AC — it is also exactly how small DC motors are made to spin forward and backward under microcontroller control, since reversing which diagonal pair is active reverses the current direction through the motor, and

therefore its direction of rotation. Project 2 in Section 3.2 below relies on this principle indirectly, through the relay it uses to control the motor's power path.

3.1.4 Summary: The Three Converter Families

Converter Type	Direction	Key Components	Typical Use Case
Rectifier (AC/DC)	AC → DC	Transformer, bridge rectifier (4 diodes), smoothing capacitor, voltage regulator	Wall-plug phone chargers, benchtop DC power supplies
Buck / Boost (DC/DC)	DC → DC	Switching MOSFET, inductor, diode, output capacitor	5 V → 3.3 V logic rails; battery → 5 V USB output
Inverter (DC/AC)	DC → AC	H-Bridge (4 transistor switches), oscillator/PWM control, step-up transformer	Solar inverters, UPS units, car-outlet AC adapters, motor direction control

3.2 Power Supply Sizing Guide (Step-by-Step Mathematical Example)

Understanding converter topologies answers the question ‘how does a power supply work?’ This section answers the equally important practical question every project eventually faces: ‘how big a power supply do I actually need?’ A supply sized for the wrong number — or sized using the wrong kind of number — is one of the most common causes of unreliable IoT and automation projects, so this section walks through a complete, professional sizing calculation using this program's Project 2: Simple Motor Protection System as the worked example.

3.2.1 Why Steady-State Current Is the Wrong Number to Design Around

A common beginner mistake is to size a power supply using only a component's normal, steady-state ‘running’ current draw. Industrial power-supply sizing guidance — including Eaton's published sizing methodology for control-panel power supplies — makes clear that a power supply must be sized around two separate demands, not one: the **inrush (surge) load**, the brief current spike a device draws the instant it is energized or forced to a stop, and the **sealed (nominal, steady-state) load**, the ongoing current it draws once running normally. Both figures are calculated by simply summing every simultaneous load's contribution, and the supply must be able to survive the higher of the two demands, not just the number a component's datasheet lists as its ‘typical’ draw.

In plain terms

Sizing a power supply around steady-state current alone is like renting a bus based only on how many seats are filled once everyone has calmly found a place to sit — while ignoring the crush of people all trying to board through the door at exactly the same moment. The bus (the power supply) has to survive that crowded doorway moment without buckling, even though it only lasts a second or two, or the whole trip never gets underway.

For a DC motor, this inrush moment happens twice: once at startup, when the motor's windings briefly draw several times their running current before they build up speed, and again any time the motor is mechanically blocked or 'stalled' — for instance, if a jammed mechanism prevents it from turning. A motor held at stall can draw close to its full stall current continuously, which is exactly the scenario Project 2's motor protection logic exists to detect and shut down. Because that stall condition is both realistic and sustained, it — not the brief startup spike — is the correct peak figure to design the power budget around.

3.2.2 Hardware Architecture: Isolating Logic from the Motor Power Loop

Before any sizing math is done, it is worth being explicit about *where* the current in this example actually flows, because Project 2's hardware is deliberately split into two electrically separate loops that must never be allowed to merge carelessly:

- **The Logic Loop (ESP32 side):** the ESP32 itself, its 3.3 V regulator, and any sensors it reads run on a small, clean, low-current supply. This side is sensitive — voltage spikes, electrical noise, or a sudden current surge here can corrupt the microcontroller's program or damage the chip outright.
- **The Power Loop (motor side):** the DC motor and the relay that switches it draw far more current, and a motor winding is inductive — meaning it can generate voltage spikes of its own the instant it is switched off. This loop is deliberately fed from its own external DC power source, sized independently from the ESP32's own supply.
- **The Relay as the Isolation Boundary:** an electromechanical relay is the component that lets the low-power logic loop control the high-power motor loop without ever physically connecting them. The ESP32 energizes only the relay's low-current control coil; internally, that coil pulls a mechanical switch closed on a completely separate set of contacts that carry the motor's current. No electrical path exists between the ESP32's GPIO pin and the motor's power path — only a magnetic one, inside the relay itself.

Why this matters

Keeping these two loops electrically separate is not a nicety — it is what keeps a stalled motor, a voltage spike from a switched-off inductive coil, or a wiring fault on the power side from ever reaching the ESP32's delicate 3.3 V logic pins. The relay's physical, mechanically-switched contacts are the barrier that makes this isolation possible; sizing the motor's power supply independently from the ESP32's own supply, as this section does next, is a direct consequence of keeping that barrier intact.

3.2.3 Project 2 Load Inventory: Simple Motor Protection System

The external power rail in Project 2 — the rail this sizing exercise is calculating — has exactly two simultaneous loads on it whenever the relay is energized and the motor is running: the DC motor itself, and the relay's own control coil.

Component	Normal Run Current	Stall / Peak Current
DC Motor (5 V)	300 mA	1,200 mA (1.2 A)
Electromechanical Relay Coil	70 mA (active coil draw)	70 mA (coil draw is essentially constant)

Both loads sit on the same external rail and can be active at the very same instant — the relay coil must stay energized to keep its contacts closed for as long as the motor is meant to be running, including the moment a stall occurs. That simultaneity is exactly why Step 1 below sums them together rather than sizing the supply around the motor alone.

3.2.4 Step 1 — Sum the Simultaneous Peak Loads

Following the same principle as Eaton's inrush-load formula ($\text{Inrush load} = \text{inrush load}_1 + \text{inrush load}_2 + \dots + \text{inrush load}_n$), the first step is simply to add up every load's worst-case current, not its typical current, because all of these loads can be drawing their peak simultaneously:

$$\text{Total Peak Load} = \text{Motor Stall Current} + \text{Relay Coil Current}$$

$$\text{Total Peak Load} = 1,200 \text{ mA} + 70 \text{ mA} = \mathbf{1,270 \text{ mA} (1.27 \text{ A})}$$

Notice that the relay coil's own current is added on top of the motor's worst-case draw, not averaged with it or ignored — in a real fault condition (a jammed shaft, for example), the motor is stalled and drawing peak current at precisely the moment the relay coil is also energized and trying to hold its contacts shut, so both numbers are simultaneously real.

3.2.5 Step 2 — Apply an Industrial Safety Margin (Headroom)

A power supply run at exactly 100% of its rated capacity has no room left for component aging, voltage sag, temperature-driven derating, or simple measurement error — and running a supply constantly at its ceiling shortens its working life through heat stress. Industrial guidance addresses this the same way Eaton's own sizing example does for its nominal load figure (multiplying steady-state demand by 120% 'for extended operational life'): by adding a fixed safety margin on top of the calculated peak. This report uses a slightly more conservative, and commonly cited, **25% headroom factor** for peak/surge loads, reflecting standard industrial thermal-safety practice:

$$\text{Required Capacity} = \text{Total Peak Load} \times \text{Safety Margin}$$

$$\text{Required Capacity} = 1.27 \text{ A} \times 1.25 = \mathbf{1.587 \text{ A}}$$

Why this matters

The 25% margin is not padding for its own sake — it is thermal and reliability headroom. A supply running near its rated ceiling generates more internal heat, ages its internal capacitors faster, and has zero buffer left the day the motor's bearings wear slightly and its real-world stall current creeps a little higher than the datasheet promised. Sizing for 1.587 A rather than the bare 1.27 A minimum is what keeps this margin available for the life of the product, not just on day one.

3.2.6 Step 3 — Select the Final Commercial Power Supply

The calculated requirement of 1.587 A is a design target, not a shopping-catalog number — commercial power adapters are manufactured in a small set of standard current ratings (0.5 A, 1 A, 1.5 A, 2 A, 3 A, and so on), so the final step is to round up to the nearest standard rating that clears the calculated requirement, never down.

Sizing Metric	Value
Step 1 — Total simultaneous peak load	1,270 mA (1.27 A)
Step 2 — Required capacity with 25% margin	1,587 mA (1.587 A)
Step 3 — Final selected power supply	5 V, 2 A regulated power supply adapter

A 5 V, 1.5 A adapter would technically fail this design — it sits below the 1.587 A requirement calculated in Step 2. A commercial **5 V, 2 A regulated power supply adapter** is the smallest standard rating that clears the requirement with margin to spare, making it the correct final specification for Project 2's external motor power rail — kept entirely separate, as established in Section 3.2.2, from whatever small regulator supplies the ESP32's own 3.3 V logic.

3.2.7 Section 3 Recap

- Rectifiers convert AC to DC using a transformer, a four-diode bridge rectifier, a smoothing capacitor, and a voltage regulator.
- Buck converters step DC voltage down and boost converters step DC voltage up, both by rapidly switching a transistor and using an inductor and capacitor to smooth the result — far more efficiently than a linear regulator.
- Inverters convert DC back to AC using an H-Bridge of four transistor switches, alternating diagonal pairs to reconstruct a reversing current — the same principle used to spin a DC motor forward and backward.
- A power supply must be sized around a load's worst-case peak (inrush/stall) current plus a safety margin, never its steady-state running current alone.
- The ESP32's logic loop and a motor's power loop must remain electrically isolated; a relay's mechanical contacts are what make that isolation possible while still letting the microcontroller control the motor circuit.
- Project 2's worked example: 1,200 mA motor stall + 70 mA relay coil = 1,270 mA peak $\times$ 1.25 margin = 1,587 mA required $\rightarrow$ a 5 V, 2 A power supply is the correct commercial selection.

References

- <https://aws.amazon.com/compare/the-difference-between-microprocessors-microcontrollers/>
- <https://www.ibm.com/think/topics/microcontroller-vs-microprocessor>
- https://zbotic.in/esp32-wroom-vs-wrover-flash-and-psram-differences-explained/?srsltid=AfmBOopzfTTW9215o7LG2HejB_TtuLpNMScbLE7O6jh3H4HfhiLmHZAp
- <https://www.innovationyourself.com/esp32-wroom-vs-esp32-wrover/>
- [**Arm Ltd.What is Instruction Set Architecture \(ISA\)?**](#)
- <https://dronebotworkshop.com/esp32-2026/>
- https://documentation.espressif.com/esp32-wrover-e_esp32-wrover-ie_datasheet_en.pdf
- https://documentation.espressif.com/esp32-wroom-32_datasheet_en.pdf
- <https://medium.com/@hardymunjhal/communication-wired-protocols-in-iot-ac263675f542>
- <https://milanachintha.medium.com/spi-communication-protocol-2b782fc03133>
- <https://thors.com/power-converters-explained-ac-to-dc-dc-to-ac-ac-to-ac-and-dc-to-dc/>
- www.eaton.com